

WAP FORGE

Tutorial da API

Como usar os modelos de IA nas suas rotinas e scripts

GUIA PARA AS ÁREAS DE NEGÓCIO · VERSÃO 1.0 · forge.wap.local

1. O que é e para que serve

O WAP Forge disponibiliza modelos de inteligência artificial hospedados dentro da empresa. Seus dados ficam na rede interna e não são enviados a serviços externos. Você usa esses modelos pelo seu próprio código (por exemplo, um script Python que resume textos, classifica comentários ou extrai informações de documentos) por meio de uma **API**, que é uma forma de um programa conversar com outro.



A API segue o mesmo padrão da OpenAI. Por isso, a biblioteca oficial `openai` para Python funciona sem alterações, bastando apontar para o endereço da empresa.

2. O que você recebe da TI

Antes de começar, a TI entrega estes itens. Anote-os aqui:

URL base	https://forge.wap.local
Sua chave (token)	sk-.....
Modelos liberados	faster, thinker, coder (confirme os da sua chave)
Validade da chave	90 dias a partir da criação (padrão)
Limite por minuto	30 requisições por minuto (padrão)
Requisições simultâneas	2 ao mesmo tempo (padrão)

Esses valores são os **padrões** aplicados às chaves novas. A TI pode ter definido valores diferentes para a sua chave; confirme com ela se precisar de mais.

> **Trate a chave como uma senha.** Ela identifica você e sua área. Não a envie por e-mail ou chat, não a grave dentro de código que será compartilhado e não a publique em repositórios. Se ela vazar, avise a TI para revogá-la e emitir outra.

3. Antes de começar

1. **Rede:** o acesso funciona apenas na rede interna da empresa ou com a **VPN ligada**. Fora dela, o endereço `forge.wap.local` não é encontrado (erro "Could not resolve host" ou "getaddrinfo failed").

2. **Python 3.9 ou superior:** confira com `python --version`.
3. **Biblioteca:** instale com `pip install openai`.

> Guarde a chave numa variável de ambiente em vez de escrevê-la no código. No Windows (PowerShell): `$env:FORGE_KEY = "sk-..."`. No Linux/macOS: `export FORGE_KEY=sk-...`.

4. Sua primeira chamada

4.1 Listar os modelos que sua chave pode usar

```
import os
from openai import OpenAI

client = OpenAI(
    base_url="https://forge.wap.local/v1",
    api_key=os.environ["FORGE_KEY"],
)

for m in client.models.list():
    print(m.id)
```

Saída esperada: `faster`, `thinker` e `coder` (um por linha).

4.2 Fazer uma pergunta

```
resposta = client.chat.completions.create(
    model="faster",
    messages=[{"role": "user", "content": "Explique em uma frase o que é uma API."}],
)
print(resposta.choices[0].message.content)
```

O campo `messages` é a conversa. Cada item tem um `role`: `user` (o que você pergunta), `system` (instruções gerais de comportamento) e `assistant` (respostas anteriores do modelo, quando você quer manter o contexto).

> **A primeira chamada pode demorar.** Se o modelo estava sem uso, ele precisa ser carregado na memória, o que pode levar dezenas de segundos. As chamadas seguintes são rápidas. Configure um tempo limite generoso: `OpenAI(..., timeout=180)`.

5. Escolhendo o modelo

Modelo	O que é	Use quando
`faster`	Modelo menor, com resposta rápida.	Tarefas simples e em volume: resumir, classificar, extrair dados, reescrever textos curtos.
`thinker`	Modelo pesado, com raciocínio (*thinking*) e suporte a ferramentas (*tools*).	Problemas que exigem análise: cálculos, comparações, decisões em várias etapas, tarefas em que o modelo precisa acionar funções suas. É mais lento.
`coder`	Modelo especializado em código.	Gerar, explicar, revisar e converter código (Python, SQL, scripts).

Na dúvida, comece pelo `faster`. Se a qualidade não bastar, tente o `thinker`.

Sobre o thinker

O `thinker` raciocina antes de responder. Esse raciocínio consome parte do limite de tokens, então use um valor de `max_tokens` alto (a partir de 4000). Com um valor baixo, a resposta pode vir vazia ou cortada. O raciocínio aparece no campo `reasoning_content` da resposta, separado do texto final em `content`.

6. Ajustando a resposta

Parâmetro	O que faz	Dica
`max_tokens`	Tamanho máximo da resposta. Um token equivale a cerca de 3/4 de uma palavra.	Sem ele, vale o padrão do modelo. No `thinker`, use 4000 ou mais.
`temperature`	Variação da resposta, de 0 a 2.	Use 0 para tarefas que exigem consistência (classificar, extrair). Valores maiores deixam o texto mais criativo.
`stream`	Recebe a resposta aos poucos, em vez de esperar o texto inteiro.	Melhor para textos longos e para telas de chat.

```
resposta = client.chat.completions.create(
    model="thinker",
    messages=[{"role": "user", "content": "Uma loja vende 3 camisetas por R$ 90. "
                                          "Quanto custam 7 camisetas? Explique em uma linha."}],
    max_tokens=4000,
    temperature=0,
)
print(resposta.choices[0].message.content)
```

Resultado obtido no teste: "Cada camiseta custa R$ 90 / 3 = R$ 30, logo 7 camisetas custam 7 x R$ 30 = **R$ 210**."

Resposta em tempo real (streaming)

```
fluxo = client.chat.completions.create(
    model="faster",
    messages=[{"role": "user", "content": "Conte de 1 a 5, separado por vírgulas."}],
    stream=True,
)
for pedaco in fluxo:
    if pedaco.choices and pedaco.choices[0].delta.content:
        print(pedaco.choices[0].delta.content, end="", flush=True)
```

7. Exemplos para o dia a dia

Em todos os exemplos abaixo, `client` é o objeto criado na seção 4.1 e `texto` é a mensagem de um cliente:

```
texto = ("O pedido 4521 foi entregue com cinco dias de atraso e a embalagem estava "
         "danificada. Já é a segunda vez que isso acontece e estou considerando "
         "cancelar o contrato.")

def perguntar(modelo, instrucao, **extra):
    r = client.chat.completions.create(
        model=modelo,
        messages=[{"role": "user", "content": instrucao + "\n\n" + texto}],
        **extra,
    )
    return r.choices[0].message.content
```

7.1 Resumir

```
print(perguntar("faster", "Resuma em uma frase:", max_tokens=200))
```

Resposta: "O pedido 4521 foi entregue com atraso e com embalagem danificada, o que já aconteceu duas vezes, levando a considerar cancelar o contrato."

7.2 Classificar

```
print(perguntar("faster",
    "Classifique o sentimento do texto como POSITIVO, NEUTRO ou NEGATIVO. "
    "Responda só com a palavra.", max_tokens=20, temperature=0))
```

Resposta: `NEGATIVO`

7.3 Extrair dados estruturados

```
print(perguntar("faster",
    "Extraia do texto, em JSON, com as chaves pedido, problemas (lista) e "
    "risco_cancelamento (true/false). Responda só com o JSON.",
    max_tokens=300, temperature=0))
```

Resposta:

```
{
  "pedido": "4521",
  "problemas": ["entregue com cinco dias de atraso", "embalagem danificada"],
  "risco_cancelamento": true
}
```

7.4 Gerar código

```
r = client.chat.completions.create(
    model="coder",
    messages=[{"role": "user", "content": "Escreva em Python uma função que recebe uma lista "
                                          "de valores e retorna a média ignorando valores None.
Só o código."}],
    max_tokens=500, temperature=0,
)
print(r.choices[0].message.content)
```

> **Confira sempre o resultado.** Os modelos podem errar ou inventar informações, principalmente em números, datas e código. Revise antes de usar em decisões ou em produção e valide o código gerado com testes.

Dicas para escrever bons pedidos (prompts)

- Seja específico sobre o formato: "responda só com a palavra", "em JSON", "em até 3 tópicos".
- Dê o contexto: quem é o leitor, qual o objetivo e o que deve ser evitado.
- Para tarefas repetitivas, mostre um ou dois exemplos do resultado esperado.
- Em lotes grandes, envie uma requisição por item e respeite o limite por minuto (seção 8).

8. Quando algo dá errado

O que você viu	Causa provável	O que fazer
"Could not resolve host" / "getaddrinfo failed"	Fora da rede interna.	Ligue a VPN e tente de novo.
401	Chave ausente, errada ou vencida.	Confira a variável `FORGE_KEY`. Se venceu, peça renovação à TI.
403	Sua chave não tem acesso ao modelo pedido.	Liste os modelos (seção 4.1) e use um da lista, ou peça a liberação.
429	Limite de requisições por minuto (padrão: 30) ou de simultâneas (padrão: 2) atingido.	Aguarde alguns segundos e tente de novo, esperando um pouco mais a cada nova tentativa.
413	Texto grande demais (acima de 5 MB).	Divida o conteúdo em partes menores.
Timeout ou erro 5xx	Modelo carregando ou servidor ocupado.	Aumente o timeout (180 s) e tente de novo.
Resposta vazia ou cortada no `thinker`	`max_tokens` baixo.	Use 4000 ou mais.

Tentar de novo automaticamente

```
client = OpenAI(
    base_url="https://forge.wap.local/v1",
    api_key=os.environ["FORGE_KEY"],
    timeout=180,
    max_retries=4,   # a biblioteca espera e repete em caso de 429 e erros temporários
)
```

9. Privacidade e boas práticas

> **Tudo é registrado.** Cada chamada grava o texto enviado, a resposta, o nome dado à sua chave (usuário, time ou grupo), o modelo usado, o consumo e o endereço de origem. Os administradores da plataforma podem consultar esses registros.

- Não envie senhas, chaves, tokens nem dados que os administradores não possam ver.

- Os dados permanecem na rede interna, mas isso não substitui as regras da empresa para dados pessoais e sigilosos. Em caso de dúvida, consulte a TI.
- Não compartilhe sua chave. Cada área deve ter a sua, para que o consumo e os limites sejam separados.
- Por padrão, a chave vale 90 dias e permite 30 requisições por minuto, no máximo 2 ao mesmo tempo. Peça renovação antes do vencimento e ajuste dos limites à TI se precisar de mais.
- Use `faster` sempre que ele bastar: é mais rápido e deixa os modelos pesados livres para quem precisa deles.

10. Suporte

Todo o suporte do WAP Forge é feito por um único contato:

Contato	Jefferson Jenzura Kopp
E-mail	jefferson.kopp@wap.ind.br

Fale com ele para:

- pedir uma chave, renovação ou revogação (por exemplo, se a chave vazar);
- aumentar limites ou liberar outro modelo;
- relatar problemas de acesso ou instabilidade.

Ao pedir ajuda, informe o nome da sua chave (não a chave em si), o modelo usado, o horário da chamada e a mensagem de erro recebida.

Referência rápida: URL base `https://forge.wap.local/v1` · Rotas: `GET /v1/models`, `POST /v1/chat/completions` · Cabeçalho: `Authorization: Bearer <sua chave>`